

Billet Flow

Complete Technical Architecture Documentation

Version: 1.0

Date: September 3, 2026

Application: CNC Estimator / Manufacturing Cost Estimation Platform

Repository: github.com/datapunchman/Billet-Flow

Billet Flow - Complete Technical Architecture Documentation

Version: 1.0

Date: September 3, 2026

Application: CNC Estimator / Manufacturing Cost Estimation Platform

Repository: <https://github.com/datapunchman/Billet-Flow>

Local Development: <http://localhost:3001>

Table of Contents

1. [Executive Summary](#)
2. [What Was Discovered](#)
3. [Current Implementation Status](#)
4. [Critical Implementation Gaps](#)
5. [Technology Stack](#)
6. [System Architecture](#)
7. [Application Route Map](#)
8. [Screen-by-Screen Architecture](#)
9. [Database Architecture](#)
10. [API Architecture](#)
11. [Authentication & Authorization](#)
12. [CNC Costing Architecture](#)
13. [Payment Architecture](#)
14. [Security Assessment](#)

- 15. [Implementation Roadmap](#)
 - 16. [Deployment Architecture](#)
 - 17. [Appendix: Screenshots](#)
-

Executive Summary

Project Overview

Billet Flow is a SaaS-based CNC manufacturing cost estimation platform that enables users to upload STEP files and receive AI-powered manufacturing cost estimates. The platform targets manufacturers, machine shops, and procurement teams who need rapid, accurate CNC machining cost calculations.

Current Development Phase

- **Status:** MVP / Alpha
- **UI/UX:** 100% Complete with production-ready design
- **Backend:** 0% Complete (Mock API only)
- **Database:** Not implemented
- **External Services:** Not integrated
- **Deployment:** Local development only

Key Findings

Strengths:

- Complete, production-ready UI/UX across 18 routes
- Consistent design system (dark navy + orange accent)
- Responsive layouts (390px to 1440px+)
- Client-side form validation
- Excellent user experience

✗ Critical Gaps:

- No backend API (100% mock data)
- No database persistence
- No real authentication/authorization
- No STEP file processing
- No CNC cost calculation engine
- No payment integration
- No email/SMS infrastructure

Production Readiness Assessment

Component	Status	Blocker Level
Frontend	✓ Ready	None
Backend API	✗ Not Started	● Critical
Database	✗ Not Started	● Critical
Authentication	✗ Mock Only	● Critical
File Processing	✗ Not Started	● Critical
Cost Engine	✗ Not Started	● Critical
Payments	✗ Not Started	● Critical
Email/SMS	✗ Not Started	● Critical

Estimated Time to Production: 6-7 months with 2-3 full-time engineers

What Was Discovered

Complete Application Crawl

18 Routes Discovered and Documented:

Public Routes (7)

1. `/` - Landing page with hero, features, pricing
2. `/register` - User registration with coupon modal
3. `/login` - User authentication
4. `/verify-email` - Email verification step
5. `/verify-phone` - Phone OTP verification (6 cells)
6. `/select-plan` - Subscription plan selection
7. `/payment` - Payment processing page

User Dashboard Routes (6)

8. `/dashboard` - User home with recent estimates
9. `/upload` - STEP file upload interface
10. `/history` - Estimate history table
11. `/estimate/[id]` - Detailed estimate view
12. `/profile` - User profile management
13. `/subscription` - Subscription management

Admin Portal Routes (6)

14. `/admin` - Admin dashboard with KPIs
15. `/admin/users` - User management table
16. `/admin/subscriptions` - Subscription management
17. `/admin/coupons` - Coupon creation and management
18. `/admin/settings` - System configuration
19. `/admin/audit` - Audit log viewer

All routes tested:  HTTP 200

Screenshots captured:  18 files at 1440x900

Technology Stack Discovery

Frontend (Current):

- Next.js 16.3.4 (App Router)
- React 19.2.8
- TypeScript 5.x
- Tailwind CSS 4.3.3
- React Hook Form + Zod validation
- Axios 1.20.0
- Zustand 5.0.15 (state management)
- Lucide React 1.40.0 (icons)
- GSAP + Anime.js (animations)

Backend (None - All Mock):

- lib/mock-api.ts with hardcoded data
- localStorage for client-side persistence
- No server-side code
- No database connections
- No external service integrations

Code Structure

```
cnc-estimator/
├─ app/                                # Next.js App Router
│  ├─ (public)/                        # Public routes
│  │  ├─ login/
│  │  ├─ register/
│  │  ├─ verify-email/
│  │  ├─ verify-phone/
│  │  ├─ select-plan/
│  │  └─ payment/
│  ├─ (authenticated)/                # User dashboard
│  │  ├─ dashboard/
│  │  ├─ upload/
│  │  ├─ history/
│  │  ├─ estimate/[id]/
│  │  ├─ profile/
│  │  └─ subscription/
│  ├─ (admin)/                        # Admin portal
│  │  └─ admin/
│  │     ├─ page.tsx                  # Dashboard
│  │     ├─ users/
│  │     ├─ subscriptions/
│  │     ├─ coupons/
│  │     ├─ settings/
│  │     └─ audit/
│  ├─ layout.tsx                      # Root layout
│  ├─ page.tsx                       # Landing page
│  └─ globals.css                    # Design system
├─ components/
│  ├─ marketing/                     # Landing components
│  └─ ui/                            # Reusable components
├─ lib/
│  ├─ api.ts                         # Axios client
│  ├─ auth.ts                        # Client auth utils
│  ├─ mock-api.ts                   # ⚠️ ALL DATA HERE
│  ├─ validations.ts               # Zod schemas
│  └─ utils.ts
└─ public/                          # Static assets
```

Current Implementation Status

Fully Implemented (UI/UX)

1. Landing Page

- Hero section with CTA
- Features grid with icons
- How it works (3-step process)
- Pricing cards (3 plans)
- FAQ accordion
- Footer with links
- Smooth scroll navigation
- Responsive design

2. Registration Flow

- Full name validation (min 2 chars)
- Email validation (proper format)
- Phone validation (+91XXXXXXXXXX)
- Password strength indicator
- Confirm password matching
- Terms checkbox requirement
- Coupon modal with backdrop
- Real-time validation errors

3. Email & Phone Verification

- Email verification page with instructions
- 6-cell OTP input with auto-advance
- Paste support for OTP codes
- Resend OTP with 60s cooldown

- Demo mode skip options
- Success state animations

4. Plan Selection

- 3 subscription plans displayed
- Pricing in INR (₹)
- Savings badges on longer plans
- Orange accent on selected plan
- Progress indicator (Step 3 of 4)
- Responsive grid layout

5. User Dashboard

- Recent estimates table
- Quick stats cards
- Upload new file CTA
- Sidebar navigation
- User menu with logout
- Responsive mobile menu

6. File Upload

- Drag and drop zone
- File type validation (.step, .stp)
- File size display
- Project name input
- Quantity selector
- Material dropdown
- Tolerance selection
- Notes textarea
- Upload progress simulation

7. Admin Dashboard

- 4 KPI cards:
 - Total Users: 1,248
 - Total Revenue: ₹1,25,840
 - Total Estimates: 5,647
 - Active Subscriptions: 456
- System health metrics
- Recent activity feed
- Sidebar navigation
- Professional B2B design

8. Admin User Management

- User table with search
- Status filters (active, trial, suspended)
- Role filters (user, admin)
- Sortable columns
- Action menu per user
- Pagination controls

9. Admin Coupon Management

- Coupon grid cards
- Usage progress bars
- Status badges (active, expired)
- Copy code button
- Create/edit/delete actions
- Expiry date display

10. Admin Settings

- General settings section
- Email settings (SMTP)
- Payment settings (Razorpay)
- Storage settings (Azure)

- AI settings (OpenAI)
- Feature flags toggles

Mock Implementation Only

lib/mock-api.ts provides:

```
// All these return hardcoded data with fake delays
mockApi.register(data)           // Always succeeds
mockApi.login(email, password)   // Any credentials work
mockApi.verifyEmail(token)       // Always succeeds
mockApi.sendPhoneOTP(phone)      // Returns success
mockApi.verifyPhoneOTP(otp)      // Accepts any 6 digits
mockApi.validateCoupon(code)     // Returns fake coupon
mockApi.uploadFile(file)         // Simulates progress
mockApi.getEstimates()           // Returns hardcoded list
mockApi.getAdminStats()          // Returns fake metrics
// ... 20+ more mock functions
```

Authentication Flow (Current):

```
User enters credentials
↓
mockApi.login() called
↓
Returns { token: "mock_" + Date.now(), user: {...} }
↓
Stored in localStorage
↓
Redirect to /admin or /dashboard
```

No validation: Any email/password combination works

No security: localStorage can be modified via console

No persistence: Data lost on page refresh

Critical Implementation Gaps

1. Backend API (DOES NOT EXIST)

Status: ● Critical Blocker

Current: All data from lib/mock-api.ts

Required: Complete REST API with Express/NestJS

Missing Endpoints (50+):

Authentication & Authorization:

POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout
POST /api/auth/refresh-token
POST /api/auth/forgot-password
POST /api/auth/reset-password
GET /api/auth/verify-email?token=
POST /api/auth/send-otp
POST /api/auth/verify-otp

User Management:

GET /api/users/me
PUT /api/users/me
PUT /api/users/me/password
DELETE /api/users/me

File Management:

POST /api/uploads # Multipart file upload
GET /api/uploads # List user uploads
GET /api/uploads/:id # Get upload details
DELETE /api/uploads/:id # Delete upload

Estimation:

POST /api/estimates # Generate new estimate
GET /api/estimates # List estimates
GET /api/estimates/:id # Get estimate detail
GET /api/estimates/:id/pdf # Download PDF

Subscription:

GET /api/subscriptions/plans # List available plans
POST /api/subscriptions # Create subscription
GET /api/subscriptions/me # Get my subscription
PUT /api/subscriptions/me/cancel
GET /api/subscriptions/me/invoices

Payment:

POST /api/payments/create-order # Razorpay order
POST /api/payments/verify-signature # Verify payment
POST /api/webhooks/razorpay # Payment webhook

Coupons:

POST /api/coupons/validate # Check coupon validity

Admin:

GET /api/admin/metrics # Dashboard KPIs

```
GET    /api/admin/users          # User management
PUT    /api/admin/users/:id
DELETE /api/admin/users/:id
POST   /api/admin/users/:id/suspend
GET    /api/admin/subscriptions
GET    /api/admin/coupons
POST   /api/admin/coupons
PUT    /api/admin/coupons/:id
DELETE /api/admin/coupons/:id
GET    /api/admin/settings
PUT    /api/admin/settings
GET    /api/admin/audit-logs
```

2. Database (DOES NOT EXIST)

Status: ● Critical Blocker

Current: No database

Required: PostgreSQL 15+

Required Tables (28+):

```
-- Core Tables
users
user_profiles
user_sessions
password_reset_tokens
email_verifications
otp_verifications

-- Subscription Tables
subscription_plans
subscriptions
subscription_history
invoices
payments

-- Coupon Tables
coupons
coupon_redemptions

-- File & Estimation Tables
uploaded_files
file_processing_jobs
estimates
estimate_items
estimate_materials
estimate_operations

-- Admin Tables
audit_logs
system_settings
feature_flags
admin_notifications

-- Supporting Tables
materials_catalog
machine_rates
labor_rates
overhead_rates
```

Example Schema:

```

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  name VARCHAR(255) NOT NULL,
  phone VARCHAR(20),
  role VARCHAR(20) DEFAULT 'user', -- user, admin
  status VARCHAR(20) DEFAULT 'trial', -- trial, active, suspended
  email_verified BOOLEAN DEFAULT FALSE,
  phone_verified BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW(),
  last_login TIMESTAMP
);

CREATE TABLE subscriptions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  plan_id VARCHAR(50) NOT NULL,
  status VARCHAR(20) NOT NULL, -- trial, active, cancelled, expired
  trial_end_date TIMESTAMP,
  billing_start_date TIMESTAMP,
  next_billing_date TIMESTAMP,
  amount_inr NUMERIC(10,2) NOT NULL,
  razorpay_subscription_id VARCHAR(100),
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE estimates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id),
  upload_id UUID REFERENCES uploaded_files(id),
  project_name VARCHAR(255) NOT NULL,
  quantity INT NOT NULL,
  material VARCHAR(100) NOT NULL,
  tolerance VARCHAR(50),
  status VARCHAR(20) DEFAULT 'processing', -- processing, completed, failed
  total_cost_inr NUMERIC(12,2),
  material_cost_inr NUMERIC(10,2),
  machining_cost_inr NUMERIC(10,2),
  labor_cost_inr NUMERIC(10,2),
  overhead_cost_inr NUMERIC(10,2),
  processing_time_seconds INT,
  machining_time_minutes NUMERIC(10,2),
  created_at TIMESTAMP DEFAULT NOW(),

```

```

        completed_at TIMESTAMP
    );

CREATE TABLE coupons (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    code VARCHAR(50) UNIQUE NOT NULL,
    description TEXT,
    discount_type VARCHAR(20) NOT NULL, -- percentage, fixed, trial_extension
    discount_value NUMERIC(10,2) NOT NULL,
    max_uses INT,
    current_uses INT DEFAULT 0,
    expires_at TIMESTAMP NOT NULL,
    applicable_plans TEXT[],
    status VARCHAR(20) DEFAULT 'active',
    created_by UUID REFERENCES users(id),
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE audit_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    actor_id UUID REFERENCES users(id),
    action VARCHAR(100) NOT NULL,
    entity_type VARCHAR(50) NOT NULL,
    entity_id UUID,
    old_value JSONB,
    new_value JSONB,
    ip_address INET,
    user_agent TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);

```

3. Authentication (MOCK ONLY)

Status: ● Critical Security Risk

Current: localStorage with fake tokens

Issues:

- ✗ No password hashing
- ✗ No JWT validation
- ✗ No token expiration
- ✗ No refresh tokens
- ✗ No HTTP-only cookies

- ✗ XSS vulnerable (localStorage)
- ✗ No CSRF protection
- ✗ Anyone can become admin via console

Required Implementation:

```

// Backend: JWT generation
import bcrypt from 'bcrypt';
import jwt from 'jsonwebtoken';

async function login(email: string, password: string) {
  // 1. Find user by email
  const user = await db.users.findUnique({ where: { email } });
  if (!user) throw new Error('Invalid credentials');

  // 2. Verify password
  const valid = await bcrypt.compare(password, user.password_hash);
  if (!valid) throw new Error('Invalid credentials');

  // 3. Generate JWT tokens
  const accessToken = jwt.sign(
    { userId: user.id, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: '15m' }
  );

  const refreshToken = jwt.sign(
    { userId: user.id },
    process.env.JWT_REFRESH_SECRET,
    { expiresIn: '7d' }
  );

  // 4. Store refresh token in database
  await db.user_sessions.create({
    data: { userId: user.id, refreshToken, expiresAt: new Date(Date.now() + 7 * 24 * 60 * 60 * 1000) }
  });

  // 5. Set HTTP-only cookies
  res.cookie('accessToken', accessToken, {
    httpOnly: true,
    secure: true,
    sameSite: 'strict',
    maxAge: 15 * 60 * 1000
  });

  res.cookie('refreshToken', refreshToken, {
    httpOnly: true,
    secure: true,
    sameSite: 'strict',
    maxAge: 7 * 24 * 60 * 60 * 1000
  });
}

```

```

    return { user: { id: user.id, email: user.email, name: user.name, role: user.role } }
  }

  // Middleware: Protect routes
  function requireAuth(req, res, next) {
    const token = req.cookies.accessToken;
    if (!token) return res.status(401).json({ error: 'Unauthorized' });

    try {
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
      req.user = decoded;
      next();
    } catch (error) {
      return res.status(401).json({ error: 'Invalid token' });
    }
  }

  function requireAdmin(req, res, next) {
    if (req.user.role !== 'admin') {
      return res.status(403).json({ error: 'Forbidden' });
    }
    next();
  }

```

4. STEP File Processing (DOES NOT EXIST)

Status: ● Critical - Core Feature

Current: Mock file upload with progress bar

Required: Complete CAD processing pipeline

Processing Pipeline:

STEP File Upload

↓

1. File Validation

- Check file extension (.step, .stp)
- Check file size (max 100MB)
- Scan for malware

↓

2. Upload to Azure Blob Storage

- Generate unique filename
- Store in user-specific container
- Generate signed URL for access

↓

3. Queue Background Job

- Add to processing queue (BullMQ)
- Return job ID to user
- Update estimate status: 'processing'

↓

4. STEP Parser (Worker Process)

- Use OpenCascade or commercial parser
- Extract geometry data
- Identify features (holes, faces, pockets, etc.)
- Calculate volume and surface area

↓

5. Feature Recognition

- Classify machining operations
- Detect holes (diameter, depth, tolerance)
- Detect pockets (dimensions, depth)
- Detect faces (area, finish requirements)
- Detect threads (type, size)

↓

6. Cost Calculation

- Material cost = volume × density × price/kg
- Machining time per feature
- Setup time estimation
- Labor cost
- Overhead allocation

↓

7. Generate Estimate Report

- Create estimate record in database
- Generate PDF report
- Store PDF in blob storage
- Update status: 'completed'

↓

8. Notify User

- Send email notification
- Update dashboard

Technology Options:

Component	Option 1	Option 2	Recommendation
STEP Parser	OpenCascade (free)	CADExchanger (paid)	Start with OpenCascade
Worker Queue	BullMQ + Redis	Azure Service Bus	BullMQ for flexibility
File Storage	Azure Blob Storage	AWS S3	Azure (consistent with hosting)
PDF Generation	Puppeteer	PDFKit	Puppeteer for complex layouts

5. CNC Cost Calculation Engine (DOES NOT EXIST)

Status: ● Critical - Core Business Logic

Current: Hardcoded estimate numbers

Required: Complete costing algorithm

Costing Formula:

$$\text{Total Cost} = \text{Material Cost} + \text{Machining Cost} + \text{Labor Cost} + \text{Overhead} + \text{Markup}$$

Where:

- $\text{Material Cost} = \text{Volume (cm}^3\text{)} \times \text{Density (g/cm}^3\text{)} \times \text{Material Price (INR/kg)} \times \text{Waste Factor}$
- $\text{Machining Cost} = \sum(\text{Feature Machining Time}) \times \text{Machine Hourly Rate (INR/hr)}$
- $\text{Labor Cost} = \text{Total Machining Time} \times \text{Labor Hourly Rate (INR/hr)}$
- $\text{Overhead} = (\text{Material} + \text{Machining} + \text{Labor}) \times \text{Overhead \%}$
- $\text{Markup} = (\text{Material} + \text{Machining} + \text{Labor} + \text{Overhead}) \times \text{Markup \%}$

Material Cost Calculation:

```

interface Material {
  name: string;
  density: number;      // g/cm³
  pricePerKg: number;   // INR
  wasteFactor: number;  // 1.1 = 10% waste
}

const materials: Material[] = [
  { name: 'Aluminum 6061', density: 2.7, pricePerKg: 350, wasteFactor: 1.1 },
  { name: 'Stainless Steel 304', density: 8.0, pricePerKg: 450, wasteFactor: 1.15 },
  { name: 'Mild Steel', density: 7.85, pricePerKg: 250, wasteFactor: 1.1 },
  { name: 'Brass', density: 8.5, pricePerKg: 550, wasteFactor: 1.15 },
  { name: 'Copper', density: 8.96, pricePerKg: 650, wasteFactor: 1.2 },
];

function calculateMaterialCost(volumeCm3: number, material: Material): number {
  const massKg = (volumeCm3 * material.density) / 1000;
  const cost = massKg * material.pricePerKg * material.wasteFactor;
  return Math.round(cost * 100) / 100; // Round to 2 decimals
}

```

Machining Time Estimation:

```

interface MachiningOperation {
  type: 'drilling' | 'face-milling' | 'pocketing' | 'threading' | 'turning';
  volume?: number;      // Material removal volume (cm³)
  area?: number;        // Surface area (cm²)
  length?: number;      // Length (mm)
  diameter?: number;    // Hole/thread diameter (mm)
  depth?: number;       // Depth (mm)
  tolerance?: string;   // 'standard' | 'fine' | 'precise'
}

// Machine rates (INR per hour)
const machineRates = {
  'cnc-mill-3axis': 1200,
  'cnc-mill-5axis': 2500,
  'cnc-lathe': 1000,
  'drill-press': 500,
};

// Typical machining speeds
function estimateMachiningTime(operation: MachiningOperation): number {
  switch (operation.type) {
    case 'drilling':
      // Drilling time = depth / feed rate
      // Assume 50mm/min feed rate for standard tolerance
      const feedRate = operation.tolerance === 'precise' ? 30 : 50;
      return (operation.depth! / feedRate) + 2; // +2 min for setup

    case 'face-milling':
      // Milling time = area / (cutting width × feed rate)
      // Assume 100 cm²/min for standard
      const removalRate = operation.tolerance === 'precise' ? 60 : 100;
      return (operation.area! / removalRate) + 5; // +5 min for setup

    case 'pocketing':
      // Material removal rate varies by depth
      const volumeRate = operation.tolerance === 'precise' ? 10 : 20; // cm³/min
      return (operation.volume! / volumeRate) + 10; // +10 min for toolpath setup

    default:
      return 10; // Default estimate
  }
}

function calculateMachiningCost(operations: MachiningOperation[]): number {
  const totalMinutes = operations.reduce((sum, op) => sum + estimateMachiningTime(op),
  const machineRate = machineRates['cnc-mill-3axis']; // Default machine

```

```
const costPerMinute = machineRate / 60;  
return totalMinutes * costPerMinute;  
}
```

Complete Estimate Generation:

```

interface EstimateInput {
  volumeCm3: number;
  material: string;
  quantity: number;
  tolerance: string;
  operations: MachiningOperation[];
}

function generateEstimate(input: EstimateInput): EstimateResult {
  // 1. Material Cost
  const materialData = materials.find(m => m.name === input.material!);
  const materialCostPerUnit = calculateMaterialCost(input.volumeCm3, materialData);

  // 2. Machining Cost
  const machiningCostPerUnit = calculateMachiningCost(input.operations);

  // 3. Labor Cost (assume 1 operator per machine)
  const laborRate = 400; // INR per hour
  const totalMachiningMinutes = input.operations.reduce((sum, op) => sum + estimateMach
  const laborCostPerUnit = (totalMachiningMinutes / 60) * laborRate;

  // 4. Setup Cost (one-time, amortized over quantity)
  const setupTime = 60; // 1 hour setup
  const setupCost = (setupTime / 60) * (machineRates['cnc-mill-3axis'] + laborRate);
  const setupCostPerUnit = setupCost / input.quantity;

  // 5. Subtotal
  const subtotalPerUnit = materialCostPerUnit + machiningCostPerUnit + laborCostPerUnit

  // 6. Overhead (20%)
  const overheadPerUnit = subtotalPerUnit * 0.20;

  // 7. Markup (quantity-based discount)
  let markupPercent = 0.30; // 30% default
  if (input.quantity >= 100) markupPercent = 0.15;
  else if (input.quantity >= 50) markupPercent = 0.20;
  else if (input.quantity >= 10) markupPercent = 0.25;

  const markupPerUnit = subtotalPerUnit * markupPercent;

  // 8. Total
  const totalPerUnit = subtotalPerUnit + overheadPerUnit + markupPerUnit;
  const totalForQuantity = totalPerUnit * input.quantity;

  return {
    perUnit: {

```

```

        material: materialCostPerUnit,
        machining: machiningCostPerUnit,
        labor: laborCostPerUnit,
        setup: setupCostPerUnit,
        overhead: overheadPerUnit,
        markup: markupPerUnit,
        total: totalPerUnit
    },
    total: totalForQuantity,
    quantity: input.quantity,
    currency: 'INR',
    leadTimeDays: Math.ceil(totalMachiningMinutes * input.quantity / (8 * 60)) // 8-hou
};
}

```

Missing Components:

- ❌ Volume calculation from STEP file
- ❌ Feature recognition algorithm
- ❌ Material properties database
- ❌ Machining operation classification
- ❌ Setup time estimation
- ❌ Tool selection logic
- ❌ Surface finish requirements
- ❌ Quantity-based pricing tiers

6. Payment Integration (DOES NOT EXIST)

Status: ● Critical - Revenue Blocker

Current: Mock payment page

Required: Razorpay integration for INR payments

Razorpay Implementation:

```

// Backend: Create Razorpay order
import Razorpay from 'razorpay';

const razorpay = new Razorpay({
  key_id: process.env.RAZORPAY_KEY_ID,
  key_secret: process.env.RAZORPAY_KEY_SECRET
});

async function createSubscriptionOrder(userId: string, planId: string, couponCode?: string) {
  // 1. Get plan details
  const plan = await db.subscription_plans.findUnique({ where: { id: planId } });

  // 2. Apply coupon if provided
  let amount = plan.price_inr;
  if (couponCode) {
    const coupon = await validateAndApplyCoupon(couponCode, userId, planId);
    if (coupon.discount_type === 'percentage') {
      amount = amount * (1 - coupon.discount_value / 100);
    } else if (coupon.discount_type === 'fixed') {
      amount = amount - coupon.discount_value;
    }
  }

  // 3. Create Razorpay order
  const order = await razorpay.orders.create({
    amount: Math.round(amount * 100), // Convert to paise
    currency: 'INR',
    receipt: `order_${userId}_${Date.now()}`,
    notes: {
      userId,
      planId,
      couponCode: couponCode || null
    }
  });

  // 4. Store order in database
  await db.payment_orders.create({
    data: {
      id: order.id,
      user_id: userId,
      plan_id: planId,
      amount_inr: amount,
      currency: 'INR',
      status: 'created',
      razorpay_order_id: order.id
    }
  });
}

```

```

});

return {
  orderId: order.id,
  amount,
  currency: 'INR',
  key: process.env.RAZORPAY_KEY_ID
};
}

// Frontend: Razorpay checkout
async function handlePayment() {
  // 1. Create order on backend
  const order = await api.post('/api/payments/create-order', {
    planId: selectedPlan,
    couponCode
  });

  // 2. Open Razorpay checkout
  const options = {
    key: order.data.key,
    amount: order.data.amount * 100,
    currency: 'INR',
    name: 'DataDelimited',
    description: 'CNC Estimator Subscription',
    order_id: order.data.orderId,
    handler: async function (response: any) {
      // 3. Verify payment on backend
      const verification = await api.post('/api/payments/verify-signature', {
        orderId: order.data.orderId,
        paymentId: response.razorpay_payment_id,
        signature: response.razorpay_signature
      });

      if (verification.data.success) {
        // Payment successful
        router.push('/dashboard?payment=success');
      }
    },
    prefill: {
      name: user.name,
      email: user.email,
      contact: user.phone
    },
    theme: {
      color: '#F97316' // Orange accent
    }
  }

```

```

};

const razorpay = new (window as any).Razorpay(options);
razorpay.open();
}

// Backend: Verify payment signature
function verifyPaymentSignature(orderId: string, paymentId: string, signature: string):
    const crypto = require('crypto');
    const text = orderId + '|' + paymentId;
    const generated_signature = crypto
        .createHmac('sha256', process.env.RAZORPAY_KEY_SECRET)
        .update(text)
        .digest('hex');

    return generated_signature === signature;
}

// Backend: Payment webhook handler
async function handlePaymentWebhook(req: Request, res: Response) {
    // 1. Verify webhook signature
    const webhookSignature = req.headers['x-razorpay-signature'];
    const webhookBody = JSON.stringify(req.body);

    const expectedSignature = crypto
        .createHmac('sha256', process.env.RAZORPAY_WEBHOOK_SECRET)
        .update(webhookBody)
        .digest('hex');

    if (webhookSignature !== expectedSignature) {
        return res.status(400).json({ error: 'Invalid signature' });
    }

    // 2. Process event
    const event = req.body.event;
    const payload = req.body.payload.payment.entity;

    if (event === 'payment.captured') {
        // Payment successful
        await db.payments.update({
            where: { razorpay_payment_id: payload.id },
            data: {
                status: 'captured',
                captured_at: new Date()
            }
        });
    }
});

```

```
// Activate subscription
await activateSubscription(payload.notes.userId, payload.notes.planId);

// Send confirmation email
await sendPaymentConfirmationEmail(payload.notes.userId);
}

res.status(200).json({ received: true });
}
```

Webhook Security:

- ☒ Verify signature on every webhook
- ☒ Use idempotency keys
- ☒ Log all webhook events
- ☒ Retry failed webhooks
- ☒ Alert on payment failures

7. Email & SMS Infrastructure (DOES NOT EXIST)

Status: ● Critical - User Communication

Current: No email/SMS capability

Required: SendGrid + Twilio/MSG91 integration

Email Templates Required:

1. **Welcome Email** (after registration)
2. **Email Verification** (with token link)
3. **OTP Email** (if SMS fails)
4. **Estimate Ready** (with PDF attachment)
5. **Payment Success** (with invoice)
6. **Subscription Renewal Reminder** (7 days before)
7. **Trial Ending** (3 days before expiration)
8. **Password Reset** (with secure link)

SendGrid Implementation:

```

import sgMail from '@sendgrid/mail';

sgMail.setApiKey(process.env.SENDGRID_API_KEY!);

async function sendEmailVerification(userId: string, email: string, name: string) {
  // 1. Generate verification token
  const token = jwt.sign({ userId, type: 'email_verification' }, process.env.JWT_SECRET!, {
    expiresIn: '24h'
  });

  // 2. Store token in database
  await db.email_verifications.create({
    data: {
      user_id: userId,
      token,
      expires_at: new Date(Date.now() + 24 * 60 * 60 * 1000)
    }
  });

  // 3. Send email
  const msg = {
    to: email,
    from: 'noreply@datadelimited.com',
    subject: 'Verify your DataDelimited account',
    templateId: 'd-xxxxxxxxxxxx', // SendGrid template ID
    dynamicTemplateData: {
      name,
      verificationUrl: `${process.env.APP_URL}/verify-email?token=${token}`
    }
  };

  await sgMail.send(msg);
}

async function sendEstimateReadyEmail(userId: string, estimateId: string) {
  const user = await db.users.findUnique({ where: { id: userId } });
  const estimate = await db.estimate.findUnique({ where: { id: estimateId } });

  // Generate PDF
  const pdfBuffer = await generateEstimatePDF(estimateId);

  const msg = {
    to: user!.email,
    from: 'noreply@datadelimited.com',
    subject: `Your estimate for ${estimate!.project_name} is ready`,
    templateId: 'd-yyyyyyyyyyyy',
  };

```

```

dynamicTemplateData: {
  name: user!.name,
  projectName: estimate!.project_name,
  totalCost: estimate!.total_cost_inr,
  estimateUrl: `${process.env.APP_URL}/estimate/${estimateId}`
},
attachments: [
  {
    content: pdfBuffer.toString('base64'),
    filename: `estimate-${estimateId}.pdf`,
    type: 'application/pdf',
    disposition: 'attachment'
  }
]
};

await sgMail.send(msg);
}

```

SMS Implementation (Twilio):

```

import twilio from 'twilio';

const client = twilio(
  process.env.TWILIO_ACCOUNT_SID,
  process.env.TWILIO_AUTH_TOKEN
);

async function sendOTP(phone: string, otp: string) {
  await client.messages.create({
    body: `Your DataDelimited verification code is: ${otp}. Valid for 5 minutes.`,
    from: process.env.TWILIO_PHONE_NUMBER,
    to: phone
  });
}

// Alternative: MSG91 for India
async function sendOTPViaMSG91(phone: string, otp: string) {
  const response = await fetch('https://api.msg91.com/api/v5/otp', {
    method: 'POST',
    headers: {
      'authkey': process.env.MSG91_AUTH_KEY!,
      'content-type': 'application/json'
    },
    body: JSON.stringify({
      template_id: process.env.MSG91_TEMPLATE_ID,
      mobile: phone.replace('+91', ''),
      otp
    })
  });
  return response.json();
}

```

OTP Generation & Validation:

```

async function generateAndSendOTP(userId: string, phone: string): Promise<void> {
  // 1. Generate 6-digit OTP
  const otp = Math.floor(100000 + Math.random() * 900000).toString();

  // 2. Hash OTP before storing
  const otpHash = await bcrypt.hash(otp, 10);

  // 3. Store in database
  await db.otp_verifications.create({
    data: {
      user_id: userId,
      phone,
      otp_hash: otpHash,
      expires_at: new Date(Date.now() + 5 * 60 * 1000), // 5 minutes
      attempts: 0
    }
  });

  // 4. Send SMS
  await sendOTP(phone, otp);
}

async function verifyOTP(userId: string, phone: string, otp: string): Promise<boolean>
// 1. Get latest OTP record
const record = await db.otp_verifications.findFirst({
  where: {
    user_id: userId,
    phone,
    verified: false,
    expires_at: { gte: new Date() }
  },
  orderBy: { created_at: 'desc' }
});

if (!record) {
  throw new Error('OTP expired or not found');
}

// 2. Check attempt limit
if (record.attempts >= 5) {
  throw new Error('Too many attempts. Please request a new OTP.');
```

```

  // 3. Verify OTP
  const valid = await bcrypt.compare(otp, record.otp_hash);

```

```

// 4. Increment attempts
await db.otp_verifications.update({
  where: { id: record.id },
  data: { attempts: record.attempts + 1 }
});

if (!valid) {
  throw new Error('Invalid OTP');
}

// 5. Mark as verified
await db.otp_verifications.update({
  where: { id: record.id },
  data: { verified: true, verified_at: new Date() }
});

// 6. Update user
await db.users.update({
  where: { id: userId },
  data: { phone_verified: true }
});

return true;
}

```

8. Object Storage (DOES NOT EXIST)

Status: ● Critical - File Persistence

Current: Files accepted but not stored

Required: Azure Blob Storage

Azure Blob Storage Implementation:

```

import { BlobServiceClient } from '@azure/storage-blob';

const blobServiceClient = BlobServiceClient.fromConnectionString(
  process.env.AZURE_STORAGE_CONNECTION_STRING!
);

async function uploadStepFile(file: Buffer, userId: string, filename: string): Promise<string> {
  // 1. Get container client
  const containerName = 'step-files';
  const containerClient = blobServiceClient.getContainerClient(containerName);

  // 2. Create unique blob name
  const timestamp = Date.now();
  const blobName = `${userId}/${timestamp}-${filename}`;
  const blockBlobClient = containerClient.getBlockBlobClient(blobName);

  // 3. Upload file
  await blockBlobClient.uploadData(file, {
    blobHTTPHeaders: { blobContentType: 'application/octet-stream' }
  });

  // 4. Generate SAS URL (valid for 7 days)
  const expiresOn = new Date();
  expiresOn.setDate(expiresOn.getDate() + 7);

  const sasUrl = await blockBlobClient.generateSasUrl({
    permissions: 'r',
    expiresOn
  });

  // 5. Store metadata in database
  await db.uploaded_files.create({
    data: {
      user_id: userId,
      filename,
      blob_name: blobName,
      blob_url: sasUrl,
      file_size_bytes: file.length,
      uploaded_at: new Date()
    }
  });

  return sasUrl;
}

async function getFileUrl(fileId: string, userId: string): Promise<string> {

```

```
const file = await db.uploaded_files.findFirst({
  where: { id: fileId, user_id: userId }
});

if (!file) {
  throw new Error('File not found');
}

// Generate new SAS URL if expired
const containerClient = blobServiceClient.getContainerClient('step-files');
const blockBlobClient = containerClient.getBlockBlobClient(file.blob_name);

const expiresOn = new Date();
expiresOn.setHours(expiresOn.getHours() + 1); // 1 hour validity

return await blockBlobClient.generateSasUrl({
  permissions: 'r',
  expiresOn
});
}
```

File Lifecycle Management:

```
// Delete files after 90 days
async function cleanupOldFiles() {
  const cutoffDate = new Date();
  cutoffDate.setDate(cutoffDate.getDate() - 90);

  const oldFiles = await db.uploaded_files.findMany({
    where: {
      uploaded_at: { lte: cutoffDate }
    }
  });

  const containerClient = blobServiceClient.getContainerClient('step-files');

  for (const file of oldFiles) {
    // Delete from blob storage
    const blockBlobClient = containerClient.getBlockBlobClient(file.blob_name);
    await blockBlobClient.deleteIfExists();

    // Delete from database
    await db.uploaded_files.delete({ where: { id: file.id } });
  }
}
```

9. Background Job Processing (DOES NOT EXIST)

Status: ● Critical - Async Operations

Current: No background processing

Required: BullMQ + Redis

BullMQ Setup:

```

import { Queue, Worker } from 'bullmq';
import Redis from 'ioredis';

const connection = new Redis({
  host: process.env.REDIS_HOST,
  port: parseInt(process.env.REDIS_PORT || '6379'),
  password: process.env.REDIS_PASSWORD
});

// Define queues
const stepProcessingQueue = new Queue('step-processing', { connection });
const emailQueue = new Queue('email', { connection });
const reportGenerationQueue = new Queue('report-generation', { connection });

// Add job to queue
async function queueStepFileProcessing(uploadId: string, userId: string) {
  await stepProcessingQueue.add('process-step-file', {
    uploadId,
    userId
  }, {
    attempts: 3,
    backoff: {
      type: 'exponential',
      delay: 5000
    }
  });
}

// Worker: Process STEP files
const stepProcessingWorker = new Worker('step-processing', async (job) => {
  const { uploadId, userId } = job.data;

  try {
    // 1. Get file from blob storage
    const file = await db.uploaded_files.findUnique({ where: { id: uploadId } });
    const fileBuffer = await downloadFileFromBlob(file!.blob_name);

    // 2. Parse STEP file
    const geometryData = await parseStepFile(fileBuffer);

    // 3. Extract features
    const features = await extractFeatures(geometryData);

    // 4. Calculate cost
    const estimate = await calculateEstimate(features, userId);
  }
});

```

```

// 5. Generate PDF report
const pdfBuffer = await generatePDFReport(estimate);
const pdfUrl = await uploadPDFToBlob(pdfBuffer, userId, estimate.id);

// 6. Update database
await db.estimate.update({
  where: { id: estimate.id },
  data: {
    status: 'completed',
    pdf_url: pdfUrl,
    completed_at: new Date()
  }
});

// 7. Queue email notification
await emailQueue.add('estimate-ready', {
  userId,
  estimateId: estimate.id
});

return { success: true, estimateId: estimate.id };
} catch (error) {
  // Log error
  await db.processing_errors.create({
    data: {
      upload_id: uploadId,
      error_message: error.message,
      stack_trace: error.stack
    }
  });

  // Update estimate status
  await db.estimate.update({
    where: { upload_id: uploadId },
    data: { status: 'failed', error_message: error.message }
  });

  throw error; // Will trigger retry
}
}, { connection });

// Worker: Send emails
const emailWorker = new Worker('email', async (job) => {
  const { type, userId, estimateId } = job.data;

  if (type === 'estimate-ready') {
    await sendEstimateReadyEmail(userId, estimateId);
  }
});

```

```
}

    return { success: true };
}, { connection });

// Monitor queue health
stepProcessingQueue.on('failed', (job, err) => {
    console.error(`Job ${job?.id} failed with error ${err.message}`);
    // Send alert to admin
});
```

10. Monitoring & Logging (DOES NOT EXIST)

Status: ● Critical - Production Operations

Current: console.log only

Required: Structured logging + APM

Winston Logging:

```

import winston from 'winston';

const logger = winston.createLogger({
  level: 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }),
    winston.format.json()
  ),
  defaultMeta: { service: 'cnc-estimator-api' },
  transports: [
    new winston.transports.File({ filename: 'error.log', level: 'error' }),
    new winston.transports.File({ filename: 'combined.log' })
  ]
});

if (process.env.NODE_ENV !== 'production') {
  logger.add(new winston.transports.Console({
    format: winston.format.simple()
  }));
}

// Usage
logger.info('User registered', { userId, email });
logger.error('Payment failed', { userId, orderId, error: error.message });

```

Sentry Error Tracking:

```

import * as Sentry from '@sentry/node';

Sentry.init({
  dsn: process.env.SENTRY_DSN,
  environment: process.env.NODE_ENV,
  tracesSampleRate: 1.0
});

// Catch all errors
app.use(Sentry.Handlers.errorHandler());

```

Security Assessment

Critical Vulnerabilities

Vulnerability	Severity	Exploitation	Mitigation
No server-side authentication	 Critical	Anyone can modify localStorage to gain admin access	Implement JWT with HTTP-only cookies
No API validation	 Critical	No API exists, but when built, must validate all inputs	Server-side Zod/Joi validation
No rate limiting	 Critical	Brute force attacks possible	Redis-based rate limiting
No CSRF protection	 Critical	Cross-site request forgery	CSRF tokens required
Secrets in UI	 Medium	API keys visible in settings page	Move to environment variables + vault
No XSS protection	 Medium	User input not sanitized	DOMPurify + Content Security Policy
No SQL injection protection	 Critical	When database added, must use parameterized queries	Use ORM (Prisma) with prepared statements

Security Checklist for Production

Authentication & Authorization:

- ☐ Implement JWT with refresh tokens
- ☐ Use HTTP-only, secure cookies
- ☐ Hash passwords with bcrypt (10+ rounds)

- ☐ Implement role-based access control (RBAC)
- ☐ Add route guards on backend
- ☐ Verify JWT on every API request
- ☐ Implement token rotation
- ☐ Add session management
- ☐ Log all authentication events
- ☐ Implement account lockout after N failed attempts

API Security:

- ☐ Validate all inputs with Zod/Joi
- ☐ Sanitize user input (DOMPurify)
- ☐ Implement rate limiting (Redis)
- ☐ Add CORS configuration
- ☐ Use helmet.js for HTTP headers
- ☐ Implement request size limits
- ☐ Add API versioning
- ☐ Use HTTPS only (redirect HTTP)
- ☐ Implement CSP headers
- ☐ Add API documentation (Swagger)

Data Security:

- ☐ Enable PostgreSQL encryption at rest
- ☐ Use TLS 1.3 for all connections
- ☐ Encrypt PII in database
- ☐ Use Azure Key Vault for secrets
- ☐ Implement audit logging
- ☐ Add data retention policies
- ☐ Implement GDPR compliance (data export/delete)
- ☐ Use prepared statements (SQL injection prevention)
- ☐ Validate file uploads (type, size, malware scan)

- ☐ Generate secure random tokens (crypto.randomBytes)

Infrastructure Security:

- ☐ Use Azure Front Door with WAF
 - ☐ Enable DDoS protection
 - ☐ Implement IP whitelisting for admin
 - ☐ Use private endpoints for database
 - ☐ Enable Azure Security Center
 - ☐ Implement backup and disaster recovery
 - ☐ Use separate environments (dev/staging/prod)
 - ☐ Rotate secrets regularly
 - ☐ Enable audit logging on all services
 - ☐ Implement intrusion detection
-

Implementation Roadmap

Phase 1: Backend Foundation (Weeks 1-6)

Week 1-2: Core Infrastructure

- ☐ Set up NestJS project structure
- ☐ Configure TypeScript + ESLint + Prettier
- ☐ Set up PostgreSQL database (Azure)
- ☐ Design database schema (28+ tables)
- ☐ Implement Prisma ORM
- ☐ Create database migrations
- ☐ Set up Redis for caching
- ☐ Configure environment variables
- ☐ Set up logging (Winston)

- ☐ Set up error tracking (Sentry)

Week 3-4: Authentication & Authorization

- ☐ Implement user registration API
- ☐ Implement login API with JWT
- ☐ Implement refresh token mechanism
- ☐ Hash passwords with bcrypt
- ☐ Implement email verification flow
- ☐ Implement OTP generation and verification
- ☐ Implement forgot password flow
- ☐ Create RBAC middleware
- ☐ Create route guards (user/admin)
- ☐ Add session management

Week 5-6: Core User APIs

- ☐ GET /api/users/me - Get current user
- ☐ PUT /api/users/me - Update profile
- ☐ PUT /api/users/me/password - Change password
- ☐ DELETE /api/users/me - Delete account
- ☐ GET /api/users/me/subscription - Get subscription
- ☐ Add input validation (Zod)
- ☐ Add rate limiting
- ☐ Write unit tests
- ☐ Write integration tests
- ☐ API documentation (Swagger)

Deliverables:

- ☒ Working backend API
- ☒ Database with all tables
- ☒ Authentication system

- ☒ User profile management
- ☒ Test coverage: 80%+

Phase 2: File Processing (Weeks 7-10)

Week 7-8: Storage & Upload

- ☐ Set up Azure Blob Storage
- ☐ Implement file upload API
- ☐ Validate file types (.step, .stp only)
- ☐ Validate file size (max 100MB)
- ☐ Generate signed URLs
- ☐ Implement multipart upload
- ☐ Store file metadata in database
- ☐ Implement file lifecycle management
- ☐ Add file deletion endpoint
- ☐ Test with large files (100MB)

Week 9-10: STEP Processing

- ☐ Integrate STEP file parser (OpenCascade)
- ☐ Implement feature extraction
- ☐ Calculate volume and surface area
- ☐ Detect holes (diameter, depth)
- ☐ Detect pockets (dimensions)
- ☐ Detect faces (area)
- ☐ Set up BullMQ job queue
- ☐ Create background worker
- ☐ Implement job retry logic
- ☐ Add processing error handling

Deliverables:

- ☒ File upload working

- ☒ STEP parser integration
- ☒ Feature extraction
- ☒ Background job processing

Phase 3: Cost Calculation (Weeks 11-16)

Week 11-12: Costing Engine Foundation

- ☐ Create materials catalog
- ☐ Implement material cost calculation
- ☐ Create machine rates database
- ☐ Implement machining time estimation
- ☐ Create labor rates configuration
- ☐ Implement labor cost calculation
- ☐ Implement setup cost calculation
- ☐ Test with sample parts

Week 13-14: Advanced Costing

- ☐ Implement overhead allocation
- ☐ Implement quantity-based pricing
- ☐ Implement discount logic
- ☐ Handle currency (INR)
- ☐ Implement tax calculation
- ☐ Add tolerance-based pricing
- ☐ Implement material waste factor
- ☐ Test edge cases

Week 15-16: Estimate Generation

- ☐ POST /api/estimates - Create estimate
- ☐ GET /api/estimates - List estimates
- ☐ GET /api/estimates/:id - Get details
- ☐ Implement PDF report generation

- ☐ Generate itemized breakdown
- ☐ Store PDF in blob storage
- ☐ GET /api/estimates/:id/pdf - Download
- ☐ Add estimate search and filters

Deliverables:

- ☒ Complete costing engine
- ☒ Estimate generation
- ☒ PDF report generation
- ☒ Cost accuracy: $\pm 5\%$

Phase 4: Payment Integration (Weeks 17-19)

Week 17-18: Razorpay Setup

- ☐ Set up Razorpay account
- ☐ Integrate Razorpay SDK
- ☐ Implement order creation API
- ☐ Implement payment verification
- ☐ Implement subscription creation
- ☐ Handle payment success
- ☐ Handle payment failure
- ☐ Test with test credentials

Week 19: Webhooks & Edge Cases

- ☐ Implement webhook endpoint
- ☐ Verify webhook signatures
- ☐ Handle subscription activation
- ☐ Handle subscription renewal
- ☐ Handle payment refunds
- ☐ Implement idempotency
- ☐ Add retry logic

- ☐ Test webhook scenarios

Deliverables:

- ☒ Razorpay integration
- ☒ Subscription payment flow
- ☒ Webhook processing
- ☒ Invoice generation

Phase 5: Admin Portal (Weeks 20-22)

Week 20-21: Admin APIs

- ☐ GET /api/admin/metrics - Dashboard KPIs
- ☐ GET /api/admin/users - User list
- ☐ PUT /api/admin/users/:id - Edit user
- ☐ POST /api/admin/users/:id/suspend
- ☐ DELETE /api/admin/users/:id - Delete user
- ☐ GET /api/admin/subscriptions - List subscriptions
- ☐ GET /api/admin/coupons - List coupons
- ☐ POST /api/admin/coupons - Create coupon
- ☐ DELETE /api/admin/coupons/:id
- ☐ GET /api/admin/settings - System settings
- ☐ PUT /api/admin/settings - Update settings
- ☐ GET /api/admin/audit-logs - Audit trail

Week 22: Dashboard Optimization

- ☐ Optimize KPI queries
- ☐ Implement caching (5 min TTL)
- ☐ Add real-time updates (optional)
- ☐ Implement pagination
- ☐ Add export functionality (CSV)
- ☐ Performance test with 10K+ users

Deliverables:

- ☒ Complete admin API
- ☒ Real-time metrics
- ☒ Audit logging
- ☒ System settings management

Phase 6: Email & Notifications (Weeks 23-24)

Week 23: Email Infrastructure

- ☐ Set up SendGrid account
- ☐ Create email templates
- ☐ Implement email queue (BullMQ)
- ☐ Send welcome email
- ☐ Send email verification
- ☐ Send estimate ready email
- ☐ Send payment confirmation
- ☐ Send trial ending reminder
- ☐ Test email delivery

Week 24: SMS Integration

- ☐ Set up Twilio/MSG91
- ☐ Implement OTP delivery
- ☐ Handle SMS failures (fallback to email)
- ☐ Implement notification preferences
- ☐ Add unsubscribe functionality
- ☐ Test SMS delivery (Indian numbers)

Deliverables:

- ☒ Email system working
- ☒ SMS/OTP delivery

- ☒ Notification preferences
- ☒ Delivery rate: 99%+

Phase 7: Production Deployment (Weeks 25-27)

Week 25: Observability

- ☐ Set up DataDog/New Relic
- ☐ Configure APM
- ☐ Set up log aggregation
- ☐ Create dashboards
- ☐ Configure alerting rules
- ☐ Set up uptime monitoring
- ☐ Implement health check endpoints
- ☐ Configure Sentry for errors

Week 26: CI/CD & Infrastructure

- ☐ Set up GitHub Actions
- ☐ Create deployment pipeline
- ☐ Configure staging environment
- ☐ Configure production environment
- ☐ Set up database migrations
- ☐ Configure auto-scaling
- ☐ Set up CDN (Azure Front Door)
- ☐ Configure SSL certificates

Week 27: Testing & Launch

- ☐ Load testing (100+ concurrent users)
- ☐ Security audit
- ☐ Penetration testing
- ☐ Performance optimization
- ☐ Fix critical bugs

- ☐ Create runbooks
- ☐ Train support team
- ☐ Soft launch with beta users

Deliverables:

- ☒ Production environment live
 - ☒ Monitoring and alerting
 - ☒ CI/CD pipeline
 - ☒ Load tested (100+ users)
 - ☒ Security audit passed
-

Deployment Architecture

Production Infrastructure



Environment Configuration

Development

- **Frontend:** <http://localhost:3001>
- **Backend:** <http://localhost:3000>
- **Database:** Local PostgreSQL
- **Redis:** Local Redis
- **Storage:** Local file system or Azure dev

Staging

- **Frontend:** <https://staging.datadelimited.com>
- **Backend:** <https://api-staging.datadelimited.com>
- **Database:** Azure PostgreSQL (Standard)
- **Redis:** Azure Cache (Basic)
- **Storage:** Azure Blob (staging container)

Production

- **Frontend:** <https://app.datadelimited.com>
- **Backend:** <https://api.datadelimited.com>
- **Database:** Azure PostgreSQL (Premium)
- **Redis:** Azure Cache (Premium)
- **Storage:** Azure Blob (production container)

Scaling Strategy

Load Level	Users	Frontend	Backend	Database	Workers
Low	0-100	2 instances	2 instances	2 vCores	1 worker
Medium	100-1000	4 instances	5 instances	4 vCores	3 workers
High	1000-10000	10 instances	20 instances	8 vCores + replica	10 workers
Very High	10000+	Auto-scale	Auto-scale	16 vCores + replicas	Auto-scale

Appendix: Screenshots

Public Routes

1. Landing Page (<docs/architecture/screenshots/01-home.png>)

- Hero section with CTA
- Features grid
- Pricing cards
- FAQ section

2. Registration (<docs/architecture/screenshots/02-register.png>)

- Form with validation
- Password strength indicator
- Coupon modal

3. Login (<docs/architecture/screenshots/03-login.png>)

- Email and password fields

- Remember me checkbox
- Forgot password link

4. Email Verification (<docs/architecture/screenshots/04-verify-email.png>)

- Email icon
- Instructions
- Demo skip button

5. Phone Verification (<docs/architecture/screenshots/05-verify-phone.png>)

- 6 OTP input cells
- Resend OTP button
- Demo OTP hint

6. Plan Selection (<docs/architecture/screenshots/06-select-plan.png>)

- 3 plan cards
- Pricing in INR
- Savings badges

7. Payment (<docs/architecture/screenshots/07-payment.png>)

- Payment form
- Order summary

Admin Portal

8. Admin Dashboard (<docs/architecture/screenshots/08-admin-dashboard.png>)

- 4 KPI cards
- System health metrics
- Recent activity

9. User Management (<docs/architecture/screenshots/09-admin-users.png>)

- User table
- Search and filters

- Action menu

10. Subscriptions (<docs/architecture/screenshots/10-admin-subscriptions.png>)

- Subscription list
- Status filters

11. Coupons (<docs/architecture/screenshots/11-admin-coupons.png>)

- Coupon cards
- Usage progress
- Create button

12. Settings (<docs/architecture/screenshots/12-admin-settings.png>)

- System settings
- SMTP configuration
- Payment settings

13. Audit Logs (<docs/architecture/screenshots/13-admin-audit.png>)

- Activity log table
- Filters

User Dashboard

14. Dashboard (<docs/architecture/screenshots/14-user-dashboard.png>)

- Recent estimates
- Quick stats
- Upload CTA

15. Upload (<docs/architecture/screenshots/15-upload.png>)

- Drag & drop zone
- Project details form
- Material selection

16. History (<docs/architecture/screenshots/16-history.png>)

- Estimate list
- Search and filters

17. Profile (<docs/architecture/screenshots/17-profile.png>)

- User details form
- Password change

18. Subscription (<docs/architecture/screenshots/18-subscription.png>)

- Current plan
 - Usage stats
 - Billing history
-

Conclusion

Billet Flow has a **complete, production-ready UI/UX** with excellent design consistency. However, it is **not deployable to production** due to the complete absence of backend infrastructure.

Key Metrics:

- **UI Completion:** 100%
- **Backend Completion:** 0%
- **Production Readiness:** 15%
- **Estimated Time to Production:** 6-7 months
- **Estimated Cost:** 2-3 full-time engineers × 6.5 months

Critical Path:

1. Backend API + Database (6 weeks)
2. STEP Processing (4 weeks)
3. Cost Calculation (6 weeks)
4. Payment Integration (3 weeks)

5. Production Hardening (5 weeks)

Next Steps:

1. Review this documentation with stakeholders
2. Prioritize features based on business value
3. Secure budget and resources
4. Begin Phase 1: Backend Foundation
5. Establish CI/CD pipeline
6. Set up monitoring from day one

This documentation provides a complete blueprint for production implementation.

Document Version: 1.0

Last Updated: September 3, 2026

Prepared By: Claude Opus 5 (AI Assistant)

Repository: <https://github.com/datapunchman/Billet-Flow>

Contact: support@datadelimited.com